高级语言程序设计

Java补充

计算机学院 陈建全
Email: cjq@nwpu.edu.cn

2025/4/8

Java补充

- 1. Lambda表达式
- 2. 注解(Annotation)
- 3. 泛型
- 4. 反射
- 5. 集合类(自学)
- 6. 线程(自学)
- 7. 设计模式(自学)
- 8. IOC和AOP(控制反转和切面编程)

1. 1 Lambda表达式

- Lambda 表达式的概念并非起源于任何现代编程语言，而是有着深深的历史根基。
- 1. λ-演算（Lambda Calculus）
- Lambda 表达式的概念最初来自于 λ-演算，这是一种在1930年代由数学家 Alonzo Church 提出的形式化逻辑系统。λ-演算是研究函数定义、函数应用和递归的数学理论，它对于后来的编程语言设计、函数式编程和计算机科学的理论有着深远影响。
- 2. 函数式编程语言
- 在1960年代到1970年代，Lambda 表达式的概念在 Lisp 这类早期的函数式编程语言中得到实现。Lisp（LISt Processor的缩写）由 John McCarthy 创建，是第一个广泛使用 Lambda 表达式的编程语言。

- 3. Lambda 表达式进入主流编程语言
- 随着时间的发展，Lambda 表达式作为一个功能强大的构造，被集成到了更多的主流语言中，它允许在更广泛的上下文中使用匿名函数来编写更加紧凑和灵活代码。例如：
  - Python 在其早期版本中就已经支持了类似 Lambda 的匿名函数用法。
  - Java 8 在2014年引入了对 Lambda 表达式的支持，这是对 Java 编程语言重要的更新，因为它使得函数式编程风格在这个普及已久的语言中变得可行。
  - C# 在2007年随 .NET Framework 3.5的发布添加了 Lambda 表达式的支持，进一步加强了它的LINQ（语言集成查询）特性。

1. 1. Lambda表达式

- 除了Java,Pyhon, C#, 还有更多的语言支持Lambda表达式：
- C++: 从 C++11 开始，C++ 支持 Lambda 表达式，允许写出更为紧凑的代码。
- JavaScript: JavaScript 函数可以作为一等公民，并且 ES6 引入了箭头函数，它们是 Lambda 表达式的一种形式。
- Ruby: 在 Ruby 中，可以通过 lambda 或 Proc.new 创建 Lambda。
- PHP: PHP 5.3 以上版本支持匿名函数，它们是 PHP 中的 Lambda 表达式。
- Go: Go 语言中，可以使用匿名函数来实现 Lambda 表达式。
- Kotlin: Kotlin 支持 Lambda 表达式，并且在编写 Android 应用时非常常见。
- Swift: Swift 中的闭包相当于 Lambda 表达式，是 Swift 语法的核心部分。
- Scala: Scala 完全支持函数式编程，其中 Lambda 表达式是其中的重要功能之一。
- Haskell: 作为一门纯函数式编程语言，Haskell 使用 Lambda 表达式作为它函数定义的基础。

1. 2. Java的 Lambda表达式

◆Lambda 表达式允许你以匿名函数的形式将行为作为方法参数，或者将代码像数据一样传递。Lambda 表达式简化了代码编写，特别是在使用集合框架或者处理回调时。

◆Lambda 表达式的基本语法是：

```
(parameters) -> expression

(parameters) -> { statements; }
```

- 可选类型声明：不需要声明参数类型，编译器可以统一识别参数值。
- 可选的参数圆括号：一个参数无需定义圆括号，但多个参数需要定义圆括号。
- 可选的大括号：如果主体包含单个语句，则不需要使用大括号。
- 可选的返回关键字：如果主体有一个表达式返回值则编译器会自动返回值，大括号需要指明表达式返回了一个数值

1. 2. Java的 Lambda表达式

◆比如：

```
// 无参，返回 void
() -> System.out.println("Hello World")

// 接受一个参数，返回参数值的 2 倍
x -> 2 * x

// 复杂表达式
(x, y) -> {
    if(x > y) {
        return x;
    } else {
        return y;
    }
}
```

1. 3. 函数式接口

◆Lambda 表达式通常与函数式接口（Functional Interface）一起使用，函数式接口是只有一个抽象方法的接口。Java 8 中引入了 @FunctionalInterface 注解来表示一个接口是函数式接口。

◆函数式接口示例

```
@FunctionalInterface
public interface SimpleFunctionalInterface {
    void execute();
}
```

◆在这个接口上方使用 @FunctionalInterface 注解是可选的，它帮助编译器检测我们是否无意中违反了函数式接口的契约（添加了第二个抽象方法），但即便不使用这个注解，只要接口只有一个抽象方法，它仍然是一个函数式接口。

1. 3. 函数式接口

◆使用 Lambda 表达式实例化函数式接口：

```
SimpleFunctionalInterface sfi = () ->
System.out.println("Hello, Lambda!");
sfi.execute();
```

◆这里，Lambda 表达式 () -> System.out.println("Hello, Lambda!") 直接提供了 SimpleFunctionalInterface 的 execute 方法的实现。当我们调用 sfi.execute() 时，它会执行打印 "Hello, Lambda!" 到控制台。

1. 3. 函数式接口

◆一个常见的函数式接口例子是 java.util.function.Predicate，它可以配合 Lambda 表达式使用，例如：

```
Predicate<Integer> isEven = x -> x % 2 == 0;

System.out.println(isEven.test(4)); // 输出 true，因为 4 是偶数
System.out.println(isEven.test(5)); // 输出 false，因为 5 是奇数
```

1. 3. 函数式接口

◆java.util.function.Predicate的进一步演示：

```
import java.util.function.Predicate;
public class PredicateExample {
  public static void main(String[] args) {
    Predicate<Integer> isEven = number -> number % 2 == 0; //
判断一个数字是否为偶数
    Predicate<Integer> isPositive = number -> number >
0;   // 判断一个数字是否为正数
    System.out.println(isEven.test(4));  // 输出: true
    System.out.println(isEven.test(5));  // 输出: false
    System.out.println(isPositive.test(10)); // 输出: true
    System.out.println(isPositive.test(-5)); // 输出: false
    // 组合多个 Predicate
    Predicate<Integer> isEvenAndPositive =
isEven.and(isPositive);
    System.out.println(isEvenAndPositive.test(4)); //输出:true
    System.out.println(isEvenAndPositive.test(-4)); //输
出:false
    }}
```

1. 4. 方法引用

- 在 Java 中，除了使用 Lambda 表达式来实现函数式接口的方法外，还可以使用方法引用（Method References）来简化代码。
- 方法引用是用来直接引用现有方法或构造函数，它可以替代相应的 Lambda 表达式。
- 通过方法引用，我们可以在不需要指定传递参数的情况下调用方法。

1. 4. 方法引用

- 在 Java 中，有几种不同类型的方法引用，包括：
- 1. 静态方法引用（Static Method References）：
  - 指向静态方法的引用。语法为 ClassName::staticMethodName。
- 2. 实例方法引用（Instance Method References）：
  - 指向实例方法的引用。语法为 instance::instanceMethodName。
- 3. 类构造函数引用（Constructor References）：
  - 指向构造函数的引用。语法为 ClassName::new。

1. 4. 方法引用

◆静态方法引用:

■假设有一个函数式接口 MyFunctionalInterface 定义如下:

```
@FunctionalInterface
interface MyFunctionalInterface {
    void display(String message);
}
```

◆静态方法引用:

■那么可以利用静态方法引用实现这个接口:

```
class MessageUtil {
    public static void printMessage(String message) {
        System.out.println(message);
    }
}
public class Main {
    public static void main(String[] args) {
        MyFunctionalInterface functionalInterface =
MessageUtil::printMessage;
        functionalInterface.display("Hello, Method
References!");
    }
}
```

1. 4. 方法引用

◆实例方法引用:

■假设有一个类 Person，其中定义了一个实例方法 getName,引用方法如下:

```
class Person {
    public String getName() {
        return "Alice";
    }
}
```

◆实例方法引用:

- Java中Supplier<T> 是一个函数式接口，位于 java.util.function 包中。它代表一个不接受参数但返回一个结果的函数。简而言之，Supplier 用于提供某种类型的结果。
- Supplier<String> 表示一个不接受参数的方法，会返回一个 String 类型的结果。

```
public class Main {
    public static void main(String[] args) {
        Person person = new Person();

        Supplier<String> getNameSupplier = person::getName;
        System.out.println(getNameSupplier.get()); // 输出:
Alice
    }
}
```

◆构造方法引用:

■假设有一个 Person 类，它有一个带有 String 参数的构造函数:

```
public class Person {
    private String name;

    public Person(String name){
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

◆构造方法引用:

■现在，假设我们有一个函数式接口 PersonFactory 用于创建 Person 对象:

```
@FunctionalInterface
interface PersonFactory {
    Person create(String name);
}
```

◆构造方法引用:

■在 main 方法中，我们使用 Person::new 构造方法引用来实例化 PersonFactory 接口。在这个上下文中，Person::new 引用的是 Person 类的构造函数。编译器自动选择匹配 PersonFactory.create 方法签名的构造函数。

```
// 在主类中使用构造方法引用
public class Main {
    public static void main(String[] args) {
        // 通过构造方法引用来实例化PersonFactory接口
        PersonFactory factory = Person::new;

        // 使用工厂方法创建Person对象
        Person person = factory.create("Alice");

        // 输出Person对象的名字
        System.out.println(person.getName()); // 输出: Alice
    }
}
```

1. 4. 方法引用

◆方法引用（Method References）和 Lambda 表达式都是 Java 8 中引入的功能，它们都可以用来实现函数式接口的抽象方法。它们的关系可以从以下几个方面来理解：

- 简洁性： 方法引用可以被看作是一种特殊类型的 Lambda 表达式，它更加简洁。当 Lambda 表达式仅仅用来调用一个已经存在的方法时，可以用方法引用来替代。
- 功能上的等效： 从功能上来说，方法引用和对应的 Lambda 表达式是等效的。它们都不包含数据，它们都可以传递行为，而不仅仅是值。
- 语法差异：，methodName代替整个方法体。换句话说，方法引用是 Lambda 表达式的语法糖。
- 明确和可读性： 方法引用通常提供比 Lambda 表达式更清晰的代码，尤其是当方法名称清楚表达了正在执行的动作时。。

1. 5. Lambda表达式注意事项

- 变量捕获：
  - Lambda 表达式可以捕获（在 Lambda 内部使用）其外部作用域中的变量，但这些变量必须是事实上的 final，即它们在绑定后不能被修改（从 Java 8 开始，不必显式声明为 final）。
  - Lambda 表达式内部不能修改定义在其外部作用域的局部变量。
- this 关键字：
  - 在 Lambda 表达式内部，this 关键字指的是定义 Lambda 的外部类。

1. 5. Lambda表达式注意事项

- 返回值：
  - 如果 Lambda 表达式体是一个语句表达式，它就和一个返回 void 类型的方法兼容。
  - 如果 Lambda 表达式中只有一个语句，用于返回值，可以省略花括号和 return 关键字。
- 类型推断：
  - Java 编译器可以推断参数类型，这意味着在大多数情况下不需要声明参数类型。但有时为了代码清晰，你可能仍想手动指定类型。

1. 5. Lambda表达式注意事项

- 方法引用：
  - 如果 Lambda 表达式的全部内容就是调用一个已存在的方法，可以使用方法引用来表示 Lambda 表达式，例如 System.out::println。
- 局部变量：
  - 不能在 Lambda 内部声明一个与局部变量同名的参数或局部变量。

1. 6. Lambda表达式的使用场景

- Lambda 表达式可以用于各种场景，包括：
  - 作为函数的参数
  - 作为线程的执行体
  - 作为事件处理程序
  - 作为集合的过滤条件
  - 作为集合的映射函数
  - 作为集合的归约函数

1. 7. Lambda表达式和闭包(Closure)

- 在 Java 中，Lambda 表达式是一种简写语法，用于实现函数式接口（仅有一个抽象方法的接口）。Java 的 Lambda 表达式允许你将一个代码块（就是 Lambda 表达式的主体）视为一个表达式，你可以将这个表达式的结果赋给一个变量，或者将其作为参数传递给一个方法。
- 闭包是指一个函数捕获了其词法作用域中的变量。在有些编程语言中（如 JavaScript、Python 和 Ruby），这种捕捉变量的函数叫做闭包，因为它们“封闭”了它们创建时的环境。在编程上下文中，闭包通常意味着函数能够“记住”和访问该函数创建时作用域中的变量，即使那个作用域已经被移出了执行栈。
- 在 Java 中，当你在 Lambda 表达式中访问外部作用域的局部变量时，Lambda 表达式会捕捉这些变量。但 Java 有一个重要限制：只能捕获那些标记为 final 或者实际上不变的变量（事实上的 final）。这个限制略微减少了闭包的能力，因为在一些其他语言中，闭包可以修改它们捕获的变量，而 Java 不允许这样做。
- 所以，在 Java 中，虽然 Lambda 表达式具有某些闭包的特性，但由于其限制，它们并不是典型的闭包。

2. 注解(Annotation)

- 注解（Annotation）也叫元数据，是一种代码级别的说明，其实就是代码里的特殊标记，这些标记可以在编译、类加载、运行时被读取，并执行相应的处理。
- java.lang.annotation.Annotation是注解的接口，所有的注解都默认实现了此接口。
- 注解不会直接影响到程序的语义，只是作为注解（标识）存在。
- 使用注解时要在其前面增加@符号，在代码中的使用方式为“@注解名”。
- 根据注解的作用，可以将注解分为：内置注解、元数据注解（或称元注解）与自定义注解三类

内置注解

◆在JDK5.0之后，Java内置了三种注解，可以在Java程序中直接使用，将其当成一个修饰符来使用，用于修饰它支持的程序元素。

1. @Override用于限定重写父类方法，该注解只能用于方法。
2. @Deprecated用于表示某个程序元素(类，方法等)已过时。如果在其他程序中使用了此元素，则在编译时将出现警告信息。
3. @SuppressWarnings用于压制警告信息的出现

元注解

◆元注解是修饰其他注解定义的注解。Jdk5.0定义了4个标准的meta-annotation元注解类型。

1. @Target：用于描述注解的使用范围，表示该注解用于什么地方，未标注则表示该注解可用于所有的地方，可能范围值在枚举类ElemenetType中。
2. @Retention：用于描述注解在什么级别保存其信息，即注解的生命周期。可选的参数值在枚举类型 RetentionPolicy 中。
3. @Documented：用于描述修饰的注解将被javadoc工具提取成文档。
4. @Inherited: 用于描述允许子类继承父类中的注解。

◆类中使用内置注解的例子：

```
@Deprecated  // 表示prin已过时。
public void prin(String name) {
          System.out.println(name); }
@Override  //限定重写父类Object的toString()方法。如果父类没有这个方法，会报编译错误
public String toString() {
    return "Person [name=" + name + ", age=" + age + "]";   }
    public static void main(String[] args) {
    exp3_19 person = new exp3_19 ();
    person.prin("");
    @SuppressWarnings("unused")
    int i ; //因为变量i在程序中没有被使用,因此处有注解，不会有警告
    int j ;
} }
```

- 说明：
- 使用@Override注解，覆盖的方法一定要是其父类的方法，否则在Eclipse中会出现编译错误。
- 调用由@Deprecated注解的方法时，在Eclipse中会出现警告信息。
- 变量j在程序中没有被使用，又没有使用@SuppressWarnings(“unused”)注解，在Eclipse中会出现警告信息。

2.3 自定义注解

◆自 Java 5 起，开发人员就可以创建自己的自定义注解来提供类、方法、变量或参数的额外信息。注解不会直接影响代码的操作，但可以被开发工具、库和框架用来生成代码、进行静态检查或在运行时进行处理。

◆声明注解格式如下:

```
[public]@interface 注解名{
    数据类型 成员变量名称() [default 初始值];
}
```

◆@interface其相当于继承了Annotation接口

◆成员变量以无参数方法的形式来声明，即变量名称后必须有“()”

◆名称和返回值数据类型定义了该成员变量的名字和类型

◆指定变量的初始值要使用default关键字

◆比如:

```
@interface DevelopInfo {//自定义注解类,
    String author() default "彭冲";     //定义注解成员变量author, 初始值为"彭冲"
}
```

◆通过使用内置的注解来指定自定义注解的适用范围:

- @Target：可以指定注解可以应用的 Java 元素类型，例如类、方法或字段。
- @Retention：指定注解信息的保存策略，即注解在什么级别可用。
- RetentionPolicy.SOURCE：注解只在源代码中可用，编译器处理源文件时就被丢弃。
- RetentionPolicy.CLASS：编译器把注解记录在类文件中，但在运行时 VM 不需要保留注解信息。
- RetentionPolicy.RUNTIME：编译器把注解记录在类文件中，并且在运行时 VM 也会保留，因此可以通过反射获取到 注解的定义。

◆名为 MyCustomAnnotation 的注解，其中包含三个元素：author, date, 和 version。其中，author 和 version 元素有默认值，这意味着使用时可以省略。date 是必须提供的，因为它没有默认值。

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD) // 这个注解只能用于方法
public @interface MyCustomAnnotation {
    String author() default "Unknown";
    String date();
    int version() default 1;
}
```

◆注解中的 @Retention 和 @Target 元注解被用来指示自定义注解的保留策略和目标：

- @Retention(RetentionPolicy.RUNTIME) 表明该注解被保留至运行时，因此可以通过运行时反射来访问。
- @Target(ElementType.METHOD) 指示该注解只能应用于方法。

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD) // 这个注解只能用于方法
public @interface MyCustomAnnotation {
    String author() default "Unknown";
    String date();
    int version() default 1;
}
```

◆应用自定义注解

◆创建注解后，你可以开始使用它来注解方法或其他元素。例如：

```
public class MyClass {
    @MyCustomAnnotation(date = "2023-01-01")
    public void myMethod() {
        // ...
    }
}
```

◆在此，MyClass 类中的 myMethod 方法被 MyCustomAnnotation 注解，date 元素没有默认值，因此提供了值。

- 使用反射来获取注解信息
- 由于 MyCustomAnnotation 使用了 RUNTIME 保留策略，我们可以通过反射获取与方法关联的注解及其信息

2.3 自定义注解


```
import java.lang.reflect.Method;
public class AnnotationParser {
    public void parse(Class<?> clazz) throws Exception {
        for (Method method : clazz.getMethods()) {
            if (method.isAnnotationPresent(MyCustomAnnotation.class))
{
                MyCustomAnnotation annotation =
method.getAnnotation(MyCustomAnnotation.class);
                System.out.println(" 方法: " + method.getName());
                System.out.println("Author: " + annotation.author());
                System.out.println("Date: " + annotation.date());
                System.out.println("Version: " +
annotation.version());
            }
        }
    }
    public static void main(String[] args) throws Exception {
        AnnotationParser parser = new AnnotationParser();
        parser.parse(MyClass.class);
    }
}
```

◆在上述代码中，AnnotationParser 类的 parse 方法接收一个 Class 对象，遍历类中所有的方法。如果某个方法使用了 MyCustomAnnotation 注解，那么它会通过 getAnnotation 方法获取注解实例，并打印出注解的详细信息。

◆自定义注解是 Java 元编程的关键部分，被广泛用于提供如配置、数据校验和行为标注等功能。通过注解处理器和框架，注解可以有多种用途，包括代码生成、查询语言映射、依赖注入等等。

- 从JDK 5.0开始，Java引入“参数化类型（parameterized type）”的概念，这种参数化类型称为“泛型（Generic）”。
- 泛型是将数据类型参数化，即在编写代码时将数据类型定义成参数，这些类型参数在使用之前再进行指明。泛型提高了代码的重用性，使得程序更加灵活、安全和简洁。

3.1 泛型定义

◆在JDK 5.0之前，为了实现参数类型的任意化，都是通过Object类型来处理。但这种处理方式所带来的缺点是需要进行强制类型转换，此种强制类型转换不仅使代码臃肿，而且要求程序员必须对实际所使用的参数类型已知的情况下才能进行，否则容易引起ClassCastException异常。

3.1 泛型定义

- 从JDK 5.0开始，Java增加对泛型的支持。使用泛型之后就不会出现上述问题。泛型的好处是在程序编译期会对类型进行检查，捕捉类型不匹配错误，以免引起ClassCastException异常；而且泛型不需要进行强制转换，数据类型都是自动转换的。
- 泛型经常使用在类、接口和方法的定义中，分别称为泛型类、泛型接口和泛型方法。泛型类是引用类型，在内存堆中。

- **定义泛型类的语法：**

```
[访问符] class 类名<类型参数列表> {
        //类体.....
}
```

类型参数只是占位符，一般使用大写的“T”、“U”、“V”等作为类型参数

可以由多个类型参数组成，多个类型参数之间使用“,”隔开

- 【示例】泛型类：

```
class Node<T> {
    private T data;
    public Node<T> next;
    //省略......
}
```

- 在实例化泛型类时，需要指定类型参数的具体类型，例如Integer、String或一个自定义的类等。
- 实例化泛型类的语法：

```
类名<类型参数列表> 对象 = new 类名<类型参数列表> ([构造方法参数列表]);
```

- 【示例】实例化泛型类：

```
Node<String> myNode = new Node<String> ();
```

- 从Java 7开始，实例化泛型类时只需给出一对尖括号“<>”即可，Java可以推断尖括号中的泛型信息。将两个尖括号放在一起像一个菱形，因此也被称为“菱形”语法，该语法实例化泛型类的格式如下：

```
类名<类型参数列表> 对象 = new 类名<> ([构造方法参数列表]);
```

- 示例：

```
Node<String> myNode = new Node<> ();
```

- 示例代码Generic.java：

私有属性data的数据类型采用泛型

该方法显示data属性的具体类型名称

```
public class Generic<T> {
    private T data;
    public Generic(T data) {
     this.data = data;
    }
    public void showDataType() {
        System.out.println("数据的类型是:
"+data.getClass().getName());
    }
}
```

- **测试代码GenericDemo.java：**

```
Generic<String> strObj = new Generic<String>("欢迎使用泛
型类！");
strObj.showDataType();
System.out.println(strObj.getData());
System.out.println("------------------------------
");
// 定义泛型类的一个Double版本
//使用Java 7“菱形”语法实例化泛型
Generic<Double> dObj = new Generic<>(3.1415);
dObj.showDataType();
System.out.println(dObj.getData());
```

- 上述代码分别为Generic的类型形参T传入String、Double和Integer三种不同类型实参，可视为类型形参T分别被全部替换成String、Double和Integer。则产生了三种新的类型：Generic<String>、Generic<Double>和Generic<Integer>。

- **测试结果如下：**

```
数据的类型是：java.lang.String
欢迎使用泛型类！
---------------------------------
数据的类型是：java.lang.Double
3.1415
```

- 当使用一个泛型类时（包括声明泛型变量和创建泛型实例对象两种情况），都应该为此泛型类传入一个实参，否则编译器会提出泛型警告。假设现在定义一个方法，该方法的参数需要使用泛型，但类型参数是不确定的，此时如果考虑使用Object类型来解决，编译时则会出现错误。

- 以之前定义的泛型类Generic为例，考虑代码NoWildcardDemo.java：

```
// 泛型类Generic的类型参数使用Object
public static void myMethod(Generic<Object> g) {
    g.showDataType();
}
public static void main(String[] args) {
    // 参数类型是Integer
    Generic<Integer> gint = new Generic<Integer>(12);
    // 这里将产生一个错误
    myMethod(gint);
}
```

编译时产生类型不匹配的错误，程序无法运行
Generic<Integer>和
Generic<Object>是不同的类型

- 上述代码出现的问题，可以使用通配符解决；
- 通配符是由“?”来表示一个未知类型，从而解决类型被限制、不能动态根据实例进行确定的缺点。

- 下述代码使用通配符“？”重新实现上述处理过程，实现处理各种类型参数的情况， UseWildcardDemo.java 代码如下：

```
// 泛型类Generic的类型参数使用Object
public static void myMethod(Generic<?> g) {
    g.showDataType();
}
public static void main(String[] args) {
    // 参数类型是Integer
    Generic<Integer> gint = new Generic<Integer>(12);
    myMethod(gint);
}
```

- 运行结果如下：

```
数据的类型是：java.lang.Integer
```

- 泛型的类型参数可以是各种类型，但有时候需要对类型参数的取值进行一定程度的限制，以便类型参数在指定范围内。针对这种情况，Java提供了“有界类型”，来限制类型参数的取值范围。有界类型分两种：
  - *使用extends关键字声明类型参数的上界；*
  - *使用super关键字声明类型参数的下界。*

3.3 上界

- 使用extends关键字可以指定类型参数的上界，限制此类型参数必须继承 自指定的父类或父类本身。被指定的父类则称为类型参数的“上界（upper bound）”。
- 类型参数的上界可以在定义泛型时进行指定，也可以在使用泛型时进行指定。

- **语法：**

```
//定义泛型时指定类型参数的上界
[访问符] class 类名<类型参数 extends 父类> {
    //类体.....
}
//使用泛型时指定类型参数的上界
泛型类<? extends 父类>
```

- 【示例】类型参数的上界：

类型参数必须是Number类的子类（也可以是Number本身）

```
//定义泛型时指定类型参数的上界
public class Generic<T extends Number>{
      //类体......
}
//使用泛型时指定类型参数的上界
Generic<? extends Number>
```



Java中Number类是一个抽象类，所有数值类都继承此抽象类，即Integer、Long、Float、Double等用于数值操作的类都继承Number类。

- **代码UpBoundGenericDemo.java（代码1）演示使用类型参数的上界：**

```
class UpBoundGeneric<T extends Number> {
     private T data;
     public UpBoundGeneric(T data) {
          this.data = data;
     }
     public void showDataType() {
          System.out.println("数据的类型是: " +
data.getClass().getName());
     }
}
```

- **UpBoundGenericDemo.java（代码2）：**

```
public class UpBoundGenericDemo {
    // 使用泛型Generic时指定其类型参数的上界
    public static void myMethod(Generic<? extends
Number> g) {
            g.showDataType();
    }
    public static void main(String[] args) {
      Generic<Integer> gint = new Generic<Integer>(12);
      myMethod(gint);
      Generic<String> gstr = new
Generic<String>("String");
      //myMethod(gstr);
      //UpBoundGeneric<String> ubgstr = new
UpBoundGeneric<String>("指定上界");
    }
}
```

类型参数不是Number的子类时会产生错误

类型参数不是Number的子类时会产生错误

- 使用super关键字可以指定类型参数的下界，限制此类型参数必须是指定的类型本身或其父类，直至Object类。被指定的类则称为类型参数的“下界（lower bound）”。
- 类型参数的下界通常在使用泛型时进行指定，其语法格式如下所示：

```
泛型类<? super 类型>
```

- 【示例】类型参数的下界：

```
Generic<? super String>
```

- 代码UpBoundGenericDemo.java演示使用类型参数的下界：

```
// 使用泛型Generic时指定其类型参数的下界
public static void myMethod(Generic<? super String> g)
{
      g.showDataType();
}
public static void main(String[] args) {
// 参数类型是String
Generic<String> gstr = new Generic<String>("String类本身");
myMethod(gstr);
// 参数类型是Integer
Generic<Integer> gint = new Generic<Integer>(12);
// myMethod(gint);
}
```

Integer不是String本身或其父类，编译会报错



注意

泛型中使用extends关键字限制类型参数必须是指定的类本身或其子类，而super关键字限制类型参数必须是指定的类本身或其父类。在泛型中经常使用extends关键字指定上界，而很少使用super关键字指定下界。

3.4 泛型的限制

- Java语言没有真正实现泛型。Java程序在编译时生成的字节码中是不包涵泛型信息的，泛型的类型信息将在编译处理时被擦除掉，这个过程称为类型擦除。这种实现理念造成Java泛型本身有很多漏洞，虽然Java 8对类型推断进行了改进，但依然需要对泛型的使用上做一些限制，其中大多数限制都是由类型擦除和转换引起的。
- Java对泛型的限制如下：
  - 泛型的类型参数只能是类类型（包括自定义类），不能是简单类型；
  - 同一个泛型类可以有多个版本（不同参数类型），不同版本的泛型类的实例是不兼容的，例如：“Generic<String>”与“Generic<Integer>”的实例是不兼容的；

3.4 泛型的限制

- 定义泛型时，类型参数只是占位符，不能直接实例化。
  - 例如：“new T()”是错误的；
- 不能实例化泛型数组，除非是无上界的类型通配符，例如：“Generic<String> []a = new Generic<String> [10]”是错误的，而“Generic<?> []a = new Generic<?> [10]”是被允许的；
- 泛型类不能继承Throwable及其子类，即泛型类不能是异常类，不能抛出也不能捕获泛型类的异常对象，例如：“class GenericException <T> extends Exception”、“catch(T e)”都是错误的。

4. 反射

- Java 反射是一种强大的机制，允许在运行时检查类和对象以及调用其属性（包括私有的）和方法。
- 反射也可以用于动态地创建和操作数组和对象。
- 这使得 Java 程序能够在不知道具体类的情况下与任何类进行交互。
- 反射是多个 Java 核心库和框架（如 Spring 和 Hibernate）的基础。

4. 1 反射的核心组件

- Class 类： 这个类位于 java.lang 包中，每个类都有一个 Class 对象，它包含了与类有关的元数据。
- Field 类： 代表类的成员变量，可以获取和设置对象的字段值。
- Method 类： 代表类的方法，可以用来调用对象的方法。
- Constructor 类： 代表了类的构造方法，可以用来创建对象。
- Array 类： 提供了动态创建和访问 Java 数组的方法。

4.2 使用反射的例子

◆获取 Class 对象

```
Class<?> clazz = Class.forName("java.lang.String");
```

◆或者利用对象获取Class对象：

```
String s = "Hello World";
Class<?> clazz = s.getClass();
```

◆或者，已经知道了类:

```
Class<?> clazz = String.class;
```

4.2 使用反射的例子

◆创建实例

```
Class<?> clazz = Class.forName("java.util.ArrayList");
Object arrayList =
clazz.getDeclaredConstructor().newInstance();
```

◆获取和调用方法

```
Method method = clazz.getMethod("add", Object.class);
method.invoke(arrayList, "Hello World");
```

◆访问和修改字段

```
Field field = clazz.getDeclaredField("size");
field.setAccessible(true); // 对于私有字段需要这么做
int size = field.getInt(arrayList);
```

◆使用构造函数

```
Constructor<?> constructor =
clazz.getConstructor(String.class);
Object instance = constructor.newInstance("Initial Value");
```

4.3 反射注意事项

- 虽然反射 是非常强大的，允许在运行时进行类的内省（Introspection）和动态调用，但是它也有一些需要注意的不利方面：
  - 性能开销： 反射涉及到类型解析和动态调用，这些通常比直接代码执行要慢。
  - 安全限制： 反射可以访问私有成员和方法，从而打破封装原则，可能会导致安全风险。
  - 抽象泄露： 使用反射时，代码可能会变得难以理解，并且与 Java 的面向对象原则相违背。
  - 维护问题： 反射代码往往依赖于很具体的类结构，当这个结构变化时，反射代码更容易出错。
  - 编译时类型检查：使用反射编写的代码，失去了编译时类型检查的好处。如果类结构发生变化，反射代码不会显示编译错误，而是在运行时产生错误。
- 尽管有这些缺点，但反射在许多框架和库中至关重要，特别是在做泛型编程、逆向工程、测试和调试工具时。在使用时，务必要权衡其带来的利与弊，并且使用时要特别注意代码的健壮性和错误处理。